OpsPilot AI - Sprint 3: Incident History

Goal: Store analysed incidents temporarily and retrieve either all incidents or one incident using its alert ID.

What We Built

- Created **app/storage.py** for temporary in-memory storage.
- Saved every analysed alert as an incident.
- Added **GET /incidents** to retrieve all incidents.
- Added **GET /incidents/{alert_id}** to retrieve one incident.
- Added 404 error handling when an incident is not found.

Current Flow

```
POST alert -> Analyse alert -> Build incident -> Save in memory -> Retrieve all incidents or retrieve one incident by alert ID
```

storage.py

```
INCIDENTS: dict[str, dict] = {}

def save_incident(incident: dict) -> None:
  INCIDENTS[incident["alert_id"]] = incident

def get_all_incidents() -> list[dict]:
  return list(INCIDENTS.values())

def get_incident(alert_id: str) -> dict | None:
  return INCIDENTS.get(alert_id)
```

Beginner Explanation

Concept	Meaning
INCIDENTS dictionary	Temporary storage where the alert ID is the key and the complete incident is the value.
save_incident()	Adds a new incident to the INCIDENTS dictionary.
get_all_incidents()	Returns all stored incidents as a list.
get_incident()	Searches for one incident using its alert ID.
dict | None	The function may return an incident dictionary or None if it is not found.
In-memory storage	Data exists only while the application process is running.

Saving an Incident in main.py

```
incident = {
  "alert_id": str(uuid4()),
  "status": "analysed",
  "received_at": datetime.now(timezone.utc).isoformat(),
  "alert": alert.model_dump(),
  "analysis": analysis
}

save_incident(incident)
```

alert.model_dump() converts the Pydantic Alert object into a normal Python dictionary before it is stored.

GET /incidents

```
@app.get("/incidents")
def list_incidents():
  incidents = get_all_incidents()
  return {
    "total": len(incidents),
    "incidents": incidents
  }
```

This endpoint needs no input. It returns the number of incidents and the complete incident list.

GET /incidents/{alert_id}

```
@app.get("/incidents/{alert_id}")
def read_incident(alert_id: str):
  incident = get_incident(alert_id)

  if incident is None:
    raise HTTPException(status_code=404, detail="Incident not found")

  return incident
```

{alert_id} is a path parameter. The ID becomes part of the URL and identifies the required incident.

Tests Completed

Test	Result
POST /alerts - HIGH_CPU	201 Created and first incident stored.
POST /alerts - DISK_FULL	201 Created and second incident stored.
GET /incidents	200 OK with total = 2.
GET /incidents/{valid ID}	200 OK with the selected incident.
GET /incidents/wrong-id	404 Not Found with Incident not found.

Important Limitation

The storage is temporary. Restarting Uvicorn clears all incidents because they are stored in RAM, not in a database. The next sprint will add SQLite so incidents survive application restarts.

Judge Explanation

"In Sprint 3, we introduced incident history. Every analysed alert is stored and can be retrieved either as part of the complete incident list or individually using its unique alert ID. We also added proper 404 handling. The current storage is intentionally in memory, and the next step is persistent database storage."

Quick Judge and Interview Questions

Q: Why use alert ID as the dictionary key?
A: It is unique and provides fast direct lookup of one incident.

Q: What is a path parameter?
A: A dynamic value included in the URL, such as the alert ID.

Q: Why return 404?
A: The requested incident does not exist.

Q: Why use HTTPException?
A: It lets FastAPI return a proper HTTP error response.

Q: What happens after restarting the server?
A: All incidents are lost because storage is currently in memory.

Q: Why not use a database immediately?
A: In-memory storage makes the flow easier to learn before adding persistence complexity.

Sprint 3 Summary

OpsPilot AI can now store analysed incidents temporarily, list all incidents, retrieve one by ID, and return a correct 404 response when an ID is missing.